Integración de terceros

La integración de terceros de Ninox permite conectar una aplicación externa para leer catálogo y stock, y enviar pedidos hacia Ninox.

Está pensada como base para casos de uso como:

- ecommerce propio
- chatbot con catálogo y stock
- sincronización con un sistema externo
- automatizaciones comerciales

> **ESTADO DEL PROYECTO**
>
> La integración de terceros es un proyecto activamente en desarrollo. La base actual es estable para comenzar una integración y a corto plazo se irán incorporando nuevos endpoints.

Qué resuelve hoy

Hoy la integración pública cubre dos flujos principales:

1. Leer artículos, precios, categorías, etiquetas, variantes y stock.
2. Enviar pedidos desde un sistema externo a Ninox.

Si tu objetivo es desarrollar una app o integración, este es el flujo recomendado para comenzar:

1. Obtener credenciales.
2. Consumir el catálogo.
3. Normalizar los datos en tu sistema.
4. Definir una estrategia de sincronización.
5. Integrar el envío de pedidos.

Alcance y consideraciones

Antes de desarrollar, tené en cuenta estas reglas del alcance actual:

- Cada app está asociada a un solo depósito.
- Cada app está asociada a un solo punto de venta.
- El stock que recibís y los pedidos que enviás se resuelven dentro de esa configuración.
- La autenticación se realiza por token.
- Los endpoints de lectura de catálogo tienen una frecuencia mínima de consulta de 10 minutos.

Esto implica que, si tu proyecto necesita múltiples depósitos o múltiples puntos de venta, hoy debés resolverlo con integraciones separadas o coordinar el caso con Ninox.

Contratación y costo

La integración de terceros tiene un costo adicional. Antes de avanzar con el desarrollo, verificá el esquema comercial vigente en:

- ninoxnet.com/integraciones/terceros

> **IMPORTANTE**
>
> Tomá este punto en cuenta desde el inicio del proyecto, porque impacta tanto en onboarding como en la decisión de alcance de la integración.

Base URLs

Usá la base correspondiente a tu entorno:

```
Pruebas: https://api.test-ninox.com.ar
Producción: https://api.ninox.com.ar
```

Todos los endpoints de esta documentación son relativos a esa base.

Autenticación

Todas las requests deben incluir el token en el header `X-NX-TOKEN`.

```
X-NX-TOKEN: tu_token
Content-Type: application/json
```

SEGURIDAD

No expongas el token en frontend ni en aplicaciones cliente. La integración debe implementarse del lado servidor o en una capa backend intermedia.

Primer request recomendado

El punto de partida recomendado es obtener el catálogo completo:

```
curl --request GET \
  --url https://api.test-ninox.com.ar/integraciones/Terceros/GetData \
  --header 'X-NX-TOKEN: {TU_TOKEN}' \
  --header 'content-type: application/json'
```

Endpoint

GET

`/integraciones/Terceros/GetData`

Respuesta

```
Articulo[]
```

Uso típico

Con este endpoint podés:

- publicar el catálogo en un storefront
- consultar stock desde un bot
- indexar productos en un buscador
- sincronizar precios y variantes
- construir una base local para tu integración

> **FRECUENCIA DE CONSULTA**
>
> `GetData` y `GetDataCurva` tienen una frecuencia mínima de 10 minutos entre consultas consecutivas. Si consultás antes, la API puede responder `403 Forbidden`.

Modelos de lectura

La API pública ofrece dos formas de lectura de catálogo:

Catálogo estándar

GET

`/integraciones/Terceros/GetData`

Devuelve `Articulo[]`, con estructura agrupada por artículo y su curva.

Catálogo por variantes

GET

`/integraciones/Terceros/GetDataCurva`

Devuelve `ArticuloConCurva[]`, con una estructura plana que suele ser más simple para sincronizaciones, indexación y procesamiento masivo.

Flujo recomendado de implementación

1. Obtener catálogo

Consumí `GetData` y validá la respuesta real de tu integración.

2. Normalizar datos

Mapeá en tu sistema, como mínimo:

- artículo
- código
- descripción
- precios
- categorías y etiquetas
- variantes
- stock disponible

3. Definir sincronización

La estrategia más común es:

- sincronización inicial completa
- caché local o persistencia propia
- refresco periódico respetando la ventana mínima de 10 minutos
- actualización incremental mediante webhooks cuando aplique

> **SNAPSHOT COMPLETO**
>
> `GetData` debe tratarse como una foto completa del depósito asociado a la app. Cada integración debe resolver su propia lógica para detectar qué artículos o variantes dejaron de llegar y marcarlos como eliminados, o bien reemplazar el conjunto local completo y tomar a Ninox como fuente de verdad.

4. Enviar pedidos

Una vez resuelto el catálogo, integrá el flujo de creación de pedidos.

5. Agregar controles operativos

Antes de salir a producción, validá:

- reintentos por errores de red
- idempotencia para evitar pedidos duplicados
- manejo de artículos sin stock
- diferencias de precio entre sistemas
- observabilidad y logs

Pedidos

La integración permite enviar pedidos a Ninox.

POST

`/integraciones/Terceros/Pedido`

```
PedidoTerceros
```

Respuesta:

```
PedidoResultado
```

Según la configuración de tu integración, el pedido puede generar:

- un pedido de venta
- una preventa o reserva

> **VALIDACIÓN DE TOTALES**
>
> El campo `total` debe coincidir con la fórmula `subtotal + envio + recargo - descuento`.

Cancelación de pedido

POST

`/integraciones/Terceros/Pedido/cancelar`

Parámetros:

- `facturaid` (`number`): identificador del comprobante a cancelar

Respuesta:

```
NxResultado
```

Webhooks

Además de la lectura por polling, Ninox puede enviar webhooks para notificar cambios de artículos.

Esto permite:

- reducir consultas innecesarias
- acelerar la actualización de stock y precios
- mantener un catálogo local más sincronizado

La documentación específica está en Webhooks.

Esquemas de datos

La referencia de tipos y contratos está en Esquema de datos.

Qué conviene definir antes de desarrollar

Antes de implementar una app sobre esta integración, conviene tener claro:

- si tu caso necesita solo lectura o también pedidos
- si vas a trabajar con stock en tiempo real o con caché
- si tu modelo debe contemplar variantes
- dónde vas a guardar el token
- cómo vas a persistir catálogo, stock y logs
- cómo vas a manejar reintentos e idempotencia

Errores comunes

Código	Descripción	Qué revisar
`401`	Token inválido o expirado	Header `X-NX-TOKEN`
`403`	Consulta fuera de frecuencia permitida	Esperar al menos 10 minutos
`400`	Request inválida	Formato del payload o parámetros
`422`	Error de validación	Totales del pedido y datos obligatorios
`404`	Recurso no encontrado	IDs enviados en la operación
`500`	Error interno	Reintentar si corresponde y contactar soporte

Siguiente paso sugerido

Si estás arrancando una integración nueva, empezá por consumir `GetData`, validar la estructura real que recibís y recién después modelar sincronización, búsqueda y pedidos.